BCO701 – Fundamentals of IoT Communication Protocols

Experiment 1: Study of IoT Protocols

Aim:

To study and understand various IoT communication protocols including 6LoWPAN, IPv4/IPv6, Wi-Fi, Bluetooth, and MQTT.

Objective:

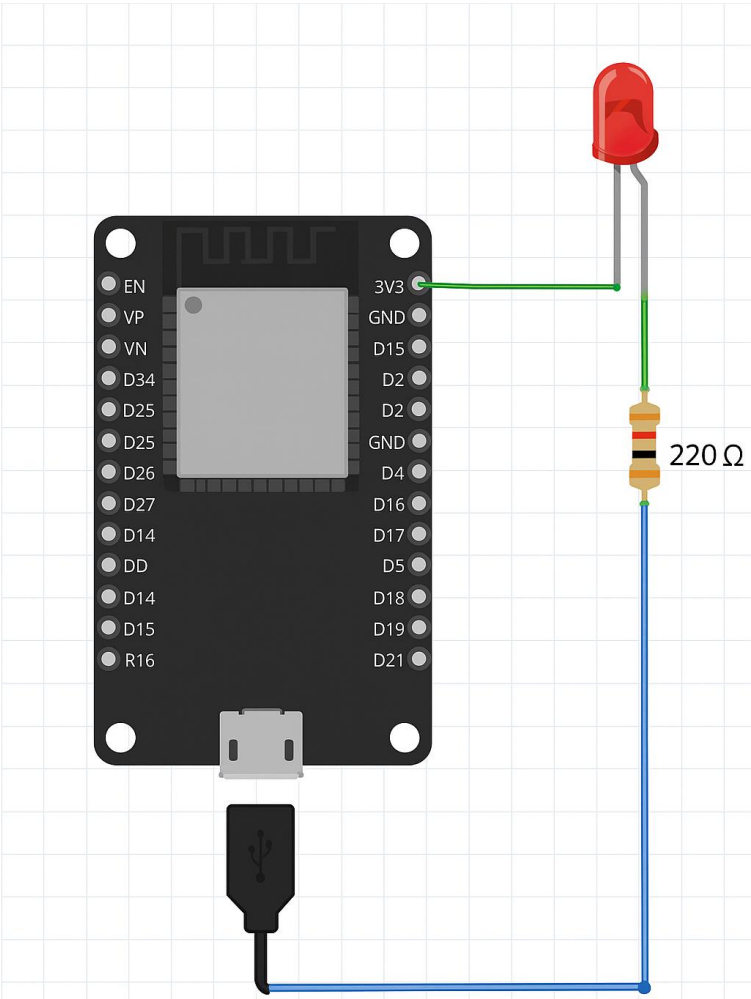
- To understand the basic concepts and structure of 6LoWPAN.
- To differentiate between IPv4 and IPv6 in IoT applications.
- To explore Wi-Fi and Bluetooth protocols for IoT connectivity.
- To implement a simple MQTT publish/subscribe model using Arduino and Node-RED.

Components Required:

Sl. No	Components	Quantity
1	ESP32/NodeMCU	1
2	USB Cable	1
3	Computer with Arduino IDE	1
4	Internet connection	1
5	MQTT Broker (e.g., HiveMQ/ Mosquitto)	1
6	LED (optional demo)	1
7	Resistor (220Ω)	1

Circuit Diagram:

Fritzing Diagram (LED control using MQTT and ESP32):



Pin ON	2 OV	Resistor	220 Ω
	GND	LED	ON

Procedure:

1. Part A: Study of Protocols (Theory)

- Study the basic architecture of each protocol.
- Understand header structure, addressing, and use-case in IoT.
- Compare IPv4 vs IPv6 for addressing limitations.
- Use online tools/documentation for protocol stack comparison.

2. Part B: Practical Implementation of MQTT Protocol

- Connect ESP32 to computer.
- Install required libraries in Arduino IDE: PubSubClient and WiFi.
- Setup MQTT broker (HiveMQ public broker or Mosquitto on localhost).
- Write Arduino code to publish LED status to MQTT topic.
- Use Node-RED/MQTT.fx to subscribe and control ESP32.
- Observe the communication and log data.

Source Code (MQTT LED Control Example):

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "Your_SSID";
const char* password = "Your_PASSWORD";
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;

WiFiClient espClient;
PubSubClient client(espClient);

const int ledPin = 2;

void setup() {
 Serial.begin(115200);
 pinMode(ledPin, OUTPUT);

 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
 }

 client.setServer(mqttServer, mqttPort);
 client.setCallback(callback);

 while (!client.connected()) {
  if (client.connect("ESP32Client")) {
   client.subscribe("iot/led");
  } else {
   delay(5000);
  }
 }
}

void loop() {
 client.loop();
}

void callback(char* topic, byte* payload, unsigned int length) {
 String message = "";
 for (int i = 0; i < length; i++) {
  message += (char)payload[i];
 }
```

```
  if (message == "ON") {
    digitalWrite(ledPin, HIGH);
  } else if (message == "OFF") {
    digitalWrite(ledPin, LOW);
  }
}
```

Input/Output:

Input: MQTT message sent to topic iot/led with payload ON or OFF.

Output: LED on ESP32 turns ON or OFF depending on message.

Observation Table:

MQTT Message Sent	LED Status on ESP32	Result
ON	ON	Success
OFF	OFF	Success

Result:

The working of various IoT protocols was studied. A demo application using the MQTT protocol was successfully implemented to control an LED using ESP32 and MQTT broker.

Viva Questions:

3. What is the role of 6LoWPAN in IoT?
4. Why is IPv6 preferred over IPv4 in IoT?
5. Compare Wi-Fi and Bluetooth in terms of power and range.
6. What is MQTT? Describe its publish/subscribe model.
7. What are QoS levels in MQTT?
8. What are some lightweight alternatives to MQTT?
9. How does 6LoWPAN compress IPv6 headers?

Experiment: Controlling the Light Emitting Diode (LED) with a Push Button

CASE 1: Using Push Button

Aim

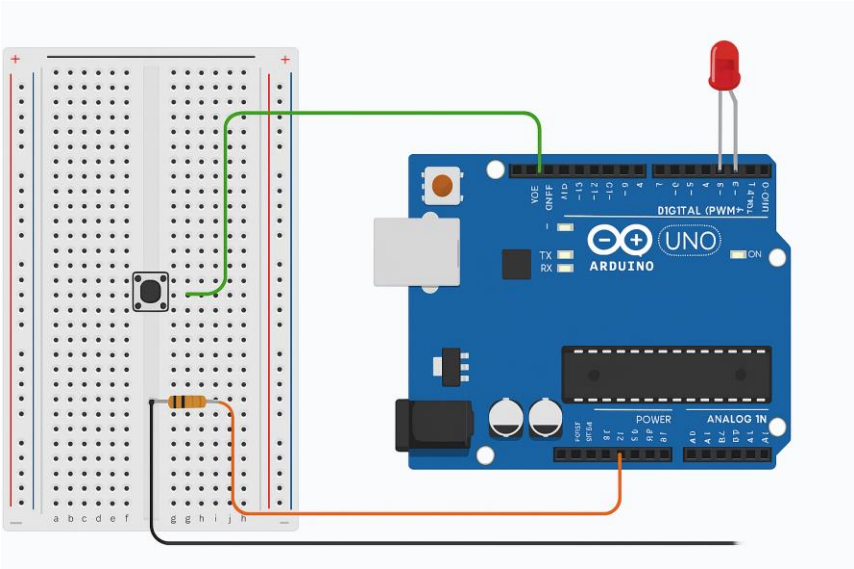
To control an LED using a push button with Arduino UNO.

Components Required

Arduino UNO
Push Button
LED
Resistor (10kΩ)
Breadboard
USB Cable
Jumper Wires

Circuit Diagram

Use Fritzing/Tinkercad to show: Push button connected to digital pin 4 with 10kΩ pull-down resistor, and LED connected to pin 13.



Code (Embedded C)

```
const int buttonPin = 4;     // Push button connected to digital pin 4
const int ledPin = 13;       // LED connected to digital pin 13

int buttonState = 0;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);
}

void loop() {
  buttonState = digitalRead(buttonPin);
  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH);
```

```
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

Result

Controlling LED by pressing the push button using Arduino UNO has been successfully execute

Experiment 3: Detection of Light Using Photoresistor (LDR)

Aim:

To detect the intensity of light using a photoresistor (LDR) and display the result using an Arduino.

Objective:

- To interface an LDR (photoresistor) with Arduino.
- To measure light levels and indicate brightness or darkness.
- To understand analog input from sensors.

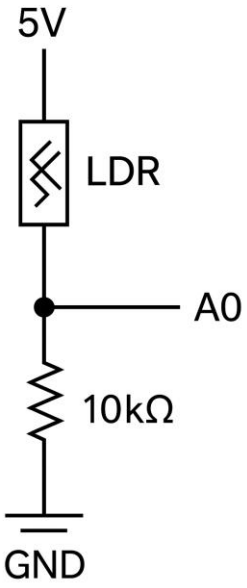
Components Required:

Component	Quantity
Arduino UNO	1
LDR (Photoresistor)	1
10kΩ Resistor	1
Breadboard	1
Jumper Wires	As required
USB Cable	1
PC with Arduino IDE	1

Circuit Diagram:

Connection Overview:
- One end of the LDR to 5V.
- The other end of the LDR to A0 (Analog pin) and also connected to GND via a 10kΩ resistor.

Fritzing Diagram:



Procedure:

1. Connect the LDR and 10kΩ resistor in a voltage divider circuit as shown.
2. Connect the junction of LDR and resistor to analog pin A0 of Arduino.
3. Upload the program to Arduino using Arduino IDE.
4. Open the Serial Monitor to observe the light intensity values.
5. Observe change in values when light is varied (e.g., cover LDR with hand).

Arduino Code:

```
// LDR Light Detection Program
int ldrPin = A0;     // LDR connected to analog pin A0
int ldrValue = 0;    // Variable to store LDR value

void setup() {
  Serial.begin(9600); // Start serial communication
  pinMode(ldrPin, INPUT);
}

void loop() {
  ldrValue = analogRead(ldrPin); // Read LDR value
```

```
  Serial.print("Light Intensity: ");
  Serial.println(ldrValue); // Display the LDR value
  delay(1000); // Wait for 1 second
}
```

Input/Output:

Light Condition	LDR Value Range (Approx.)
Bright Light	800 - 1023
Normal Light	400 - 800
Darkness/Shadow	0 - 400

Screenshot of Serial Monitor Output (Sample):

```
Light Intensity: 978
Light Intensity: 843
Light Intensity: 212
Light Intensity: 38
```

Result:

The LDR successfully detected varying light levels and printed corresponding analog values to the Serial Monitor.

Viva Questions:

1. What is the working principle of an LDR?
2. What is the range of analog values returned by analogRead() in Arduino?
3. What kind of sensor is an LDR – digital or analog?
4. What is the role of the 10kΩ resistor in this circuit?
5. How can LDR be used in a real-time IoT application?

Experiment 4: Interfacing of Temperature Sensor LM35 with Arduino

Aim:

To interface LM35 temperature sensor with Arduino and display the temperature reading on the Serial Monitor.

Objective:

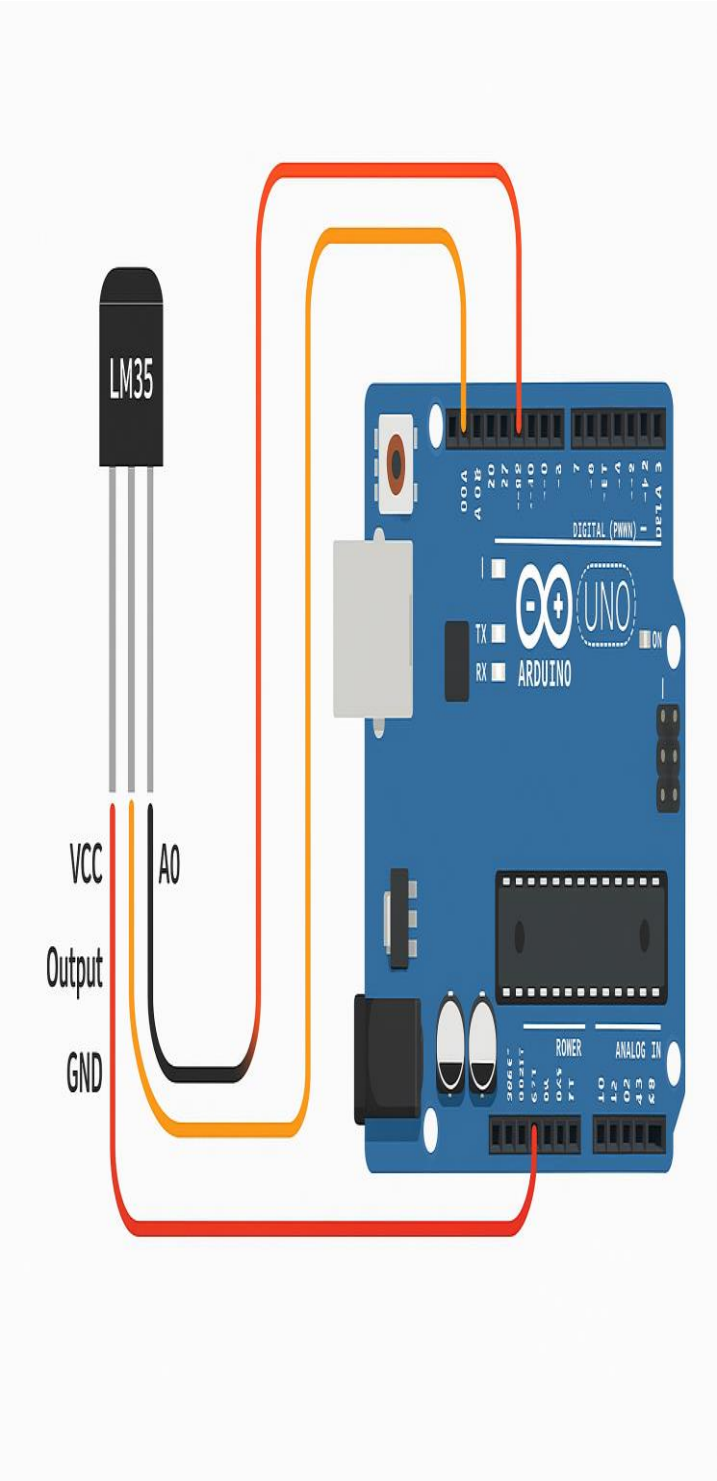
- To interface the LM35 analog temperature sensor with Arduino UNO.
- To measure room temperature.
- To understand analog sensor data acquisition using Arduino.

Components Required:

Component	Quantity
Arduino UNO	1
LM35 Temperature Sensor	1
Breadboard	1
Jumper Wires	As required
USB Cable	1
PC with Arduino IDE	1

Circuit Diagram:

Connection Overview:
- LM35 has 3 pins:
  - VCC (Pin 1) → Connect to 5V on Arduino
  - Output (Pin 2) → Connect to A0 (Analog pin on Arduino)
  - GND (Pin 3) → Connect to GND on Arduino



Circuit Diagram Image: (Insert manually here)

Procedure:

1. Place the LM35 sensor on the breadboard.
2. Connect VCC pin of LM35 to 5V on Arduino.
3. Connect GND pin of LM35 to GND on Arduino.
4. Connect output pin of LM35 to analog input pin A0 of Arduino.
5. Upload the Arduino code.

6. Open Serial Monitor to read temperature values.

Arduino Code:

```
// LM35 Temperature Sensor Interfacing

const int lm35Pin = A0; // LM35 output connected to analog pin A0
float tempC;

void setup() {
  Serial.begin(9600); // Start serial communication
}

void loop() {
  int analogValue = analogRead(lm35Pin); // Read the analog value
  tempC = (analogValue * 5.0 * 100.0) / 1024.0; // Convert to temperature in °C

  Serial.print("Temperature: ");
  Serial.print(tempC);
  Serial.println(" °C");

  delay(1000); // Wait for 1 second
}
```

Input/Output:

Analog Reading	Temperature (°C)
102	~50
205	~100
307	~150
410	~200

Sample Serial Monitor Output:

```
Temperature: 28.56 °C
Temperature: 28.64 °C
Temperature: 28.60 °C
```

Calculation Logic:

- LM35 provides 10mV per °C.
- Arduino ADC gives a value between 0–1023 for 0–5V.
- So the formula: Temperature (°C) = (analogRead × 5.0 × 100) / 1024

Result:

The LM35 temperature sensor was successfully interfaced with Arduino, and real-time temperature data was displayed on the Serial Monitor.

Viva Questions:

1. What is the sensitivity of the LM35 sensor?
2. How is temperature calculated from the analog value in LM35?
3. What is the resolution of the Arduino ADC?
4. Can LM35 measure negative temperatures?
5. Why do we multiply by 100 in the conversion formula?

EXPERIMENT – 5

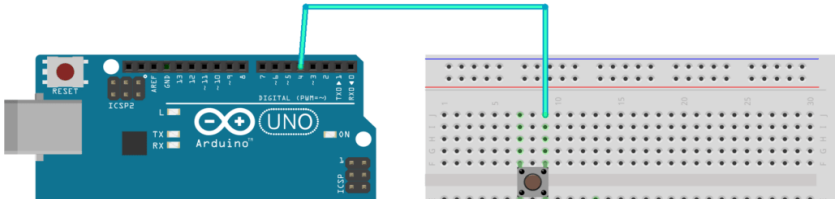
To interface motor using relay with Arduino/Raspberry Pi and write a program to turn ON motor when push button is pressed.

AIM: Write a program to turn ON motor when push button is pressed using relay with Arduino UNO.

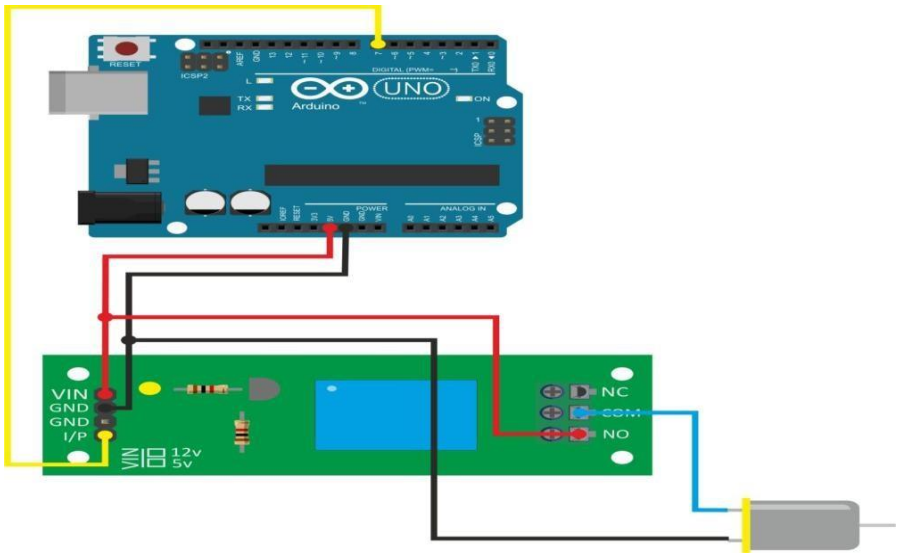
COMPONENTS REQUIRED:

1. Arduino UNO.
2. Relay.
3. Resistor (10kΩ).
4. Motor.
5. Connecting cable or USB cable.
6. Breadboard.
7. Jumper wires.

CONNECTION DIAGRAM:



(a) Push button interfacing with Arduino



(b) Interfacing motor with relay using Arduino

CODING:

Embedded C

```
#define RELAY_PIN 8 // Digital pin connected to the relay module #define
BUTTON_PIN 2 // Digital pin connected to the push button void setup()
{
  Serial.begin(9600); pinMode(RELAY_PIN,
  OUTPUT); pinMode(BUTTON_PIN,INPUT);
}
void loop()
 {
```

```
  int buttonState = digitalRead(BUTTON_PIN);


  if(buttonState == HIGH)
{
    // Button is pressed, turn ON the motor via relay
    digitalWrite(RELAY_PIN, HIGH);
    delay(1000); // Delay for motor ON time (1 second in this case)
} else
  {
    // Button is not pressed, turn OFF the motor via relay digitalWrite(RELAY_PIN, LOW);
  }
  }
```

(c) Interfacing motor with relay and push button using Arduino

RESULT: Motor turn ON/OFF operation when push button is pressed using relay with Arduino UNO has successfully executed.

Experiment: HTTP Communication with Arduino

1. Experiment Name & Aim

Name: HTTP Communication with Arduino

Aim: To develop an Arduino-based application to send sensor data to a server and receive a response using HTTP GET and POST requests.

2. Objective

- Understand HTTP protocol in IoT.
- Interface Arduino with ESP8266 Wi-Fi module.
- Send data to a server and receive response using HTTP.

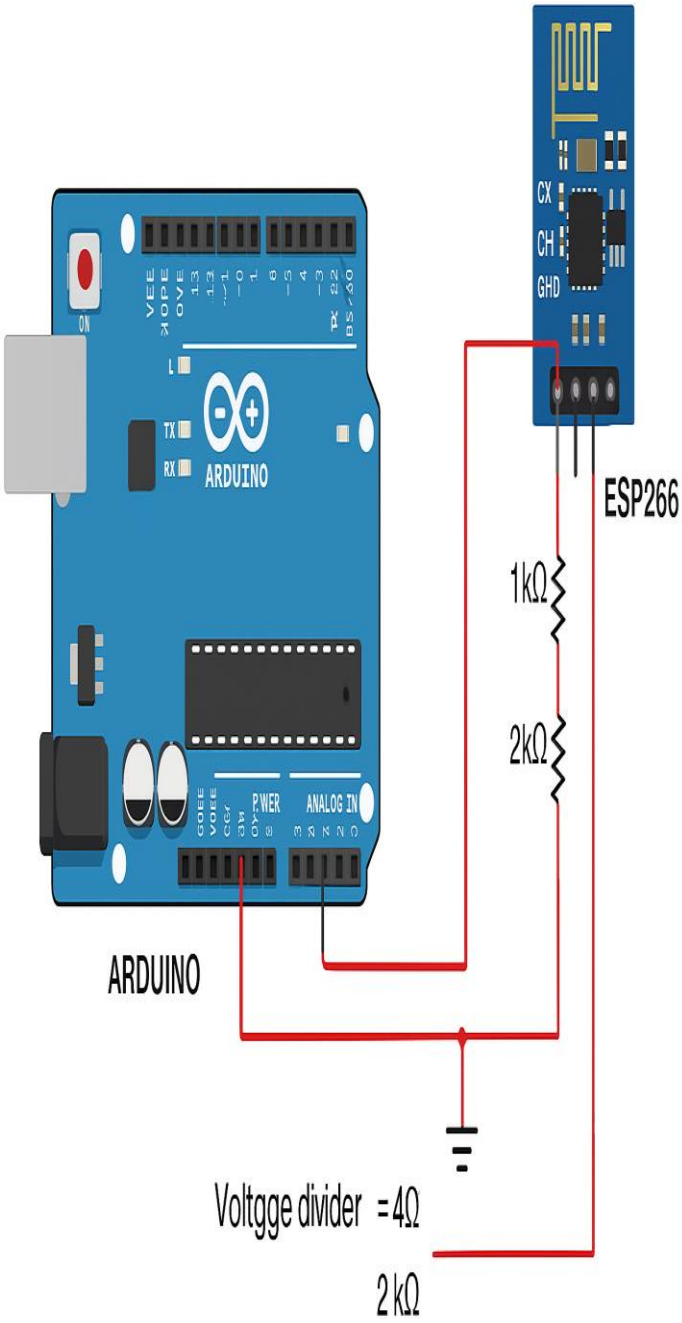
3. Components Required

Sl. No.	Components	Quantity
1	Arduino UNO	1
2	ESP8266 Wi-Fi Module	1
3	Jumper Wires	Several
4	USB Cable	1
5	Breadboard (optional)	1

4. Circuit Diagram

Connections:
ESP8266 → Arduino
TX → RX (Pin 0)
RX → TX (Pin 1) via voltage divider
VCC → 3.3V
GND → GND
CH_PD → 3.3V

(Use 1kΩ and 2kΩ voltage divider on TX line from Arduino to ESP8266 RX.)



5. Procedure

10. Connect the ESP8266 to the Arduino as per the circuit.
11. Upload the code to Arduino to send HTTP request to a web server.

12. Open Serial Monitor to view the server response.
13. Ensure that the server URL is correct and accessible.
14. Monitor real-time data exchange between Arduino and server.

6. Source Code

```
#include <SoftwareSerial.h>

SoftwareSerial esp(2, 3); // RX, TX

void setup() {
  Serial.begin(9600);
  esp.begin(9600);
  sendCommand("AT", 2000);
  sendCommand("AT+CWMODE=1", 2000);
  sendCommand("AT+CWJAP=\"Your_SSID\",\"Your_PASSWORD\"", 5000);
  sendCommand("AT+CIPSTART=\"TCP\",\"api.thingspeak.com\",80", 2000);

  String httpRequest = "GET /update?api_key=YOUR_API_KEY&field1=45\r\n";
  String cmd = "AT+CIPSEND=";
  cmd += String(httpRequest.length());
  sendCommand(cmd, 2000);
  sendCommand(httpRequest, 4000);
}

void loop() {
}

void sendCommand(String cmd, int timeout) {
  esp.println(cmd);
  long int time = millis();
  while ((time + timeout) > millis()) {
    while (esp.available()) {
      char c = esp.read();
      Serial.write(c);
    }
  }
}
```

7. Input/Output

Input: Hardcoded sensor value in HTTP request

Output: Server accepts the request and responds (shown in Serial Monitor)

8. Screenshot

```
AT
OK

AT+CWMODE=1
OK

AT+CWJAP="Your_SSID","Your_PASSWORD"
WIFI CONNECTED
WIFI GOT IP
OK

AT+CIPSTART="TCP","api.thingspeak.com",80
CONNECT
OK

AT+CIPSEND=49
>
GET /update?api_key=YOUR_API_KEY&field1=45
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 123
Server: nginx
Date: Mon, 27 Jul 2025 12:34:56 GMT

<html>
<head><title>ThingSpeak Response</title></head>
<body>
<h1>Channel Update Successful</h1>
<p>Your data has been received.</p>
</body>
</html>
OK
```

Sample Serial Monitor Screenshot:

```
AT
OK

AT+CWMODE=1
OK

AT+CWJAP="Your_SSID","Your_PASSWORD"
WIFI CONNECTED
WIFI GOT IP
OK

AT+CIPSTART="TCP","api.thingspeak.com",80
CONNECT
OK

AT+CIPSEND=49
>
GET /update?api_key=YOUR_API_KEY&field1=45
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 123
Server: nginx
Date: Mon, 27 Jul 2025 12:34:56 GMT

<html>
<head><title>ThingSpeak Response</title></head>
<body>
<h1>Channel Update Successful</h1>
<p>Your data has been received.</p>
</body>
</html>
OK
```

9. Result

The application successfully sent a HTTP GET request to a server using ESP8266 Wi-Fi and received a valid response.

10. Viva Questions

- What is HTTP protocol?
- What is the default baud rate of ESP8266?
- How is data sent using HTTP in Arduino?
- What is the role of AT commands?

- Explain the function of AT+CIPSEND.

IoT Lab Experiment: Room Temperature Monitoring and Cloud Upload

Aim

To develop an IoT application that reads temperature using LM35 sensor and uploads the data to a cloud platform (ThingSpeak) via NodeMCU (ESP8266).

Objective

- Interface LM35 sensor with NodeMCU ESP8266.
- Read temperature values.
- Upload the values to ThingSpeak cloud.

Components Required

Sl. No.	Component	Quantity
1	NodeMCU ESP8266	1
2	LM35 Temperature Sensor	1
3	Breadboard	1
4	Jumper Wires	As needed
5	USB Cable	1
6	Internet Connection	1

Circuit Diagram

LM35 to NodeMCU Connections:
- VCC → 3.3V
- GND → GND
- OUT → A0 (Analog pin)

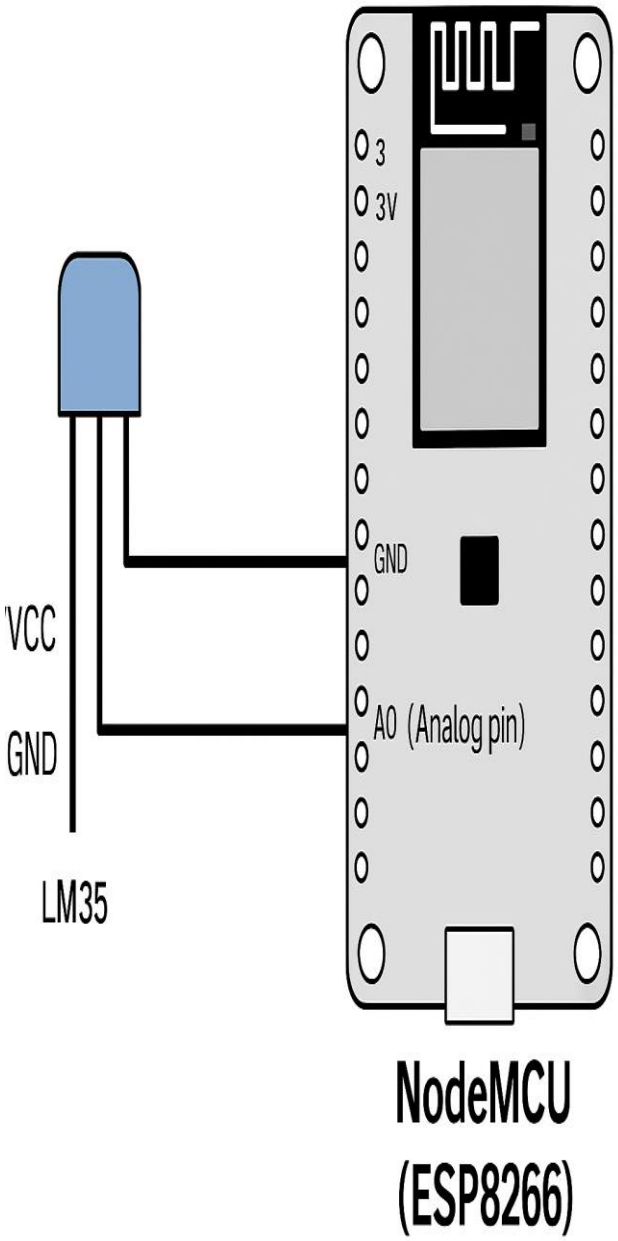


Diagram will be inserted here.

Procedure

15. Connect the LM35 sensor to NodeMCU as per the diagram.
16. Create a ThingSpeak account and a new channel with one field named “Temperature”.
17. Copy the Write API Key from ThingSpeak.

18. Open Arduino IDE.
19. Install required libraries: ESP8266WiFi, ThingSpeak.
20. Write and upload the program to NodeMCU.
21. Open Serial Monitor to view status.
22. Go to your ThingSpeak channel to see live temperature updates.

Source Code

```
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>

const char* ssid = "Your_SSID";
const char* password = "Your_PASSWORD";

WiFiClient client;

unsigned long myChannelNumber = YOUR_CHANNEL_NUMBER;
const char* myWriteAPIKey = "YOUR_API_KEY";

int lm35Pin = A0;

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");
  ThingSpeak.begin(client);
}

void loop() {
  int sensorValue = analogRead(lm35Pin);
  float voltage = sensorValue * (3.3 / 1023.0);
  float temperatureC = voltage * 100.0;

  Serial.print("Temperature: ");
  Serial.print(temperatureC);
  Serial.println(" °C");
```

```
  ThingSpeak.setField(1, temperatureC);
  int status = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);
  if (status == 200) {
    Serial.println("Data sent to ThingSpeak.");
  } else {
    Serial.println("Error sending data.");
  }
  delay(15000);
}
```

Input/Output

Input	Output
LM35 sensor	Real-time temperature on ThingSpeak

Sample Output Screenshot

Serial Monitor Output:
Connecting to WiFi....
WiFi connected
Temperature: 28.67 °C
Data sent to ThingSpeak.

Screenshot of ThingSpeak Channel can be inserted here.

Result

The application successfully reads room temperature using the LM35 sensor and uploads the value to the cloud using NodeMCU.

Viva Questions

- What is the resolution of LM35 temperature sensor?
- Why do we use 3.3V instead of 5V on ESP8266?
- What is the function of ThingSpeak?
- What is the maximum analog voltage input of NodeMCU?
- How often can we update data to ThingSpeak?
- BCO701 - Remaining Lab Programs (8 to 12)
- Experiment 8: Soil Moisture Monitoring and Upload to Firebase
- Aim:

To develop an application that measures soil moisture using a soil moisture sensor and uploads the sensed data to Google Firebase using NodeMCU (ESP8266).

Objective:
- Interface soil moisture sensor with NodeMCU.
- Read and display moisture level.
- Upload data to Firebase Cloud.

Components:
- NodeMCU ESP8266
- Soil Moisture Sensor
- Jumper wires
- Breadboard

Circuit:
- VCC → 3.3V
- GND → GND
- A0 → A0 (Analog pin on NodeMCU)

Procedure:
1. Setup Firebase real-time database and obtain Database URL and Secret Key.
2. Connect the soil sensor to NodeMCU as per diagram.
3. Upload code using Arduino IDE.
4. Data will be visible in Firebase.

Source Code:

- // Include necessary libraries
  #include <ESP8266WiFi.h>
  #include <FirebaseArduino.h>

  const char* ssid = "Your_SSID";
  const char* password = "Your_PASSWORD";

  const char* FIREBASE_HOST = "your-project.firebaseio.com";
  const char* FIREBASE_AUTH = "your_firebase_database_secret";

  int sensorPin = A0;

  void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  Firebase.begin(FIREBASE_HOST, FIREBASE_AUTH);

```
}

void loop() {
  int moisture = analogRead(sensorPin);
  Firebase.setInt("SoilMoisture", moisture);
  delay(5000);
}
```

- Experiment 9: Intrusion Detection System using Ultrasonic Sensor and Arduino
-

Aim:
To develop a simple intrusion detection system using Arduino and an ultrasonic sensor.

Objective:
- Use ultrasonic sensor to detect movement.
- Trigger alert using serial monitor or LED.

Components:
- Arduino UNO
- Ultrasonic Sensor (HC-SR04)
- Buzzer or LED

Circuit:
- VCC → 5V
- GND → GND
- Trig → Pin 9
- Echo → Pin 10

Source Code:

- ```
  #define trigPin 9
  #define echoPin 10

  void setup() {
    Serial.begin(9600);
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
  }

  void loop() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
  ```
  ```

```
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH);
  int distance = duration * 0.034 / 2;

  if (distance < 20) {
   Serial.println("Intruder detected!");
  } else {
   Serial.println("Area clear.");
  }
  delay(1000);
}
```

- Experiment 10: Directional Control of DC Motor using Arduino
-

Aim:
To control the direction of a DC motor using Arduino and L298N motor driver.

Objective:
- Interface DC motor with Arduino via L298N.
- Change direction using input pins.

Components:
- Arduino UNO
- L298N Motor Driver
- DC Motor

Circuit:
- IN1 → Pin 8
- IN2 → Pin 9
- ENA → 5V (with jumper)
- Motor to OUT1 and OUT2
- VCC → 12V battery

Source Code:

-
```
int in1 = 8;
int in2 = 9;

void setup() {
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
```

```
}

void loop() {
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  delay(2000);

  digitalWrite(in1, LOW);
  digitalWrite(in2, HIGH);
  delay(2000);
}
```

- Experiment 11: Distance Measurement and Upload to Google Cloud IoT
-

Aim:
To measure distance using an ultrasonic sensor and upload the data to Google Cloud IoT.

Objective:
- Interface ultrasonic sensor with ESP8266.
- Use HTTP/MQTT to send data to Google Cloud.

Components:
- NodeMCU ESP8266
- Ultrasonic Sensor
- Jumper Wires

(Cloud integration steps to be done using GCP MQTT bridge or Cloud Functions)

Source Code:
(Simplified)

- ```
  // (Requires GCP setup, placeholder code)
  #include <ESP8266WiFi.h>

  void setup() {
    // Initialize WiFi and sensor
  }

  void loop() {
    // Read distance
    // Send data to Google Cloud via MQTT/HTTP
  ```
  ```

```
}
```

- Experiment 12: Simple Sensor-Based Application (e.g., Temp & Humidity)
-

Aim:
To develop a basic sensor-based application using DHT11 sensor with Arduino.

Objective:
- Interface DHT11 and display readings.

Components:
- Arduino UNO
- DHT11 Sensor

Circuit:
- VCC → 5V
- GND → GND
- DATA → Pin 7

Source Code:

- ```
  #include <DHT.h>
  #define DHTPIN 7
  #define DHTTYPE DHT11

  DHT dht(DHTPIN, DHTTYPE);

  void setup() {
   Serial.begin(9600);
   dht.begin();
  }

  void loop() {
   float h = dht.readHumidity();
   float t = dht.readTemperature();
   Serial.print("Temp: "); Serial.println(t);
   Serial.print("Humidity: "); Serial.println(h);
   delay(2000);
  }
  ```
  ```